Advanced Topics of Programming Languages

1.1 - Memory Models

Example:

Thread A

```
void foo(void) {
  a = 1;
  b = 1;
}
```

Thread B

```
void bar(void) {
  while (b != 1) {};
  assert a==1;
}
```

↳ intuition: the assertion in bar should never fail.

But, given enough tries, it actually might fail!

↳ for example, some compilers may reorder the statements in foo since they don't depend on each other. Then b could be 1 but not a.

So, we need to define a memory model.

Memory interactions behave differently in the presence of

- multiple concurrent threads
- data replication in hierarchical and/or distributed memory systems
- deffered communication of updates

Memory models are a product of negotiating (basically compromising between these two)

- restrictions of freedom of implementation to guarantee race-related properties
- establishment of freedom of implementation to enable program and machine model optimisations

→ modern languages include the memory model in their language definition.

1.2 - Strict Consistency

Strict Consistency: independently of which process reads or writes, the value from the most recent write to a location is observable by reads from the respective location **immediately after the write** occurs.

↳ also, having an absolute global time is problematic

↳ although intuitively desired, it's **not physically possible!** Strict consistency is too strong to be realistic.

1.3 - Happened Before Relation

Example:

Thread A

```
void foo(void) {
    a = 1;
    b = 1;
}
```

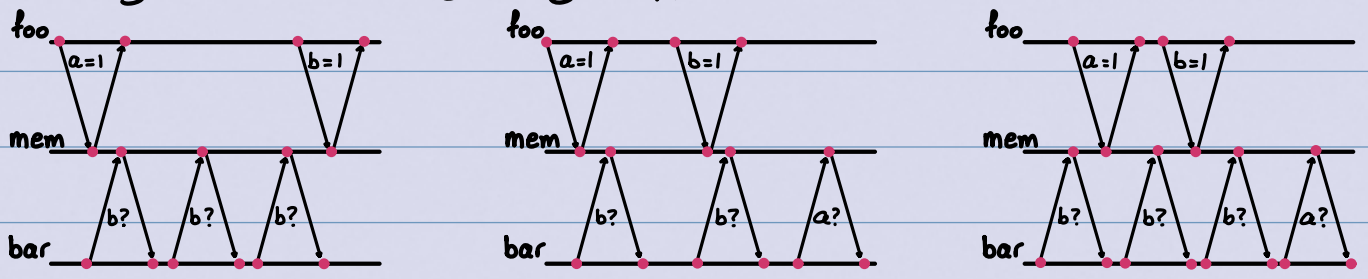
Thread B

```
void bar(void) {
    while (b != 1) {};
    assert a == 1;
}
```

- initial state of a and b is 0
- thread A stores to a before it stores to b
- thread B should see b go to 1 before executing the assert statement
- the assert statement should always hold

↳ here, correctness means: storing 1 to a **happens before** (relative-time!) loading 1 to b

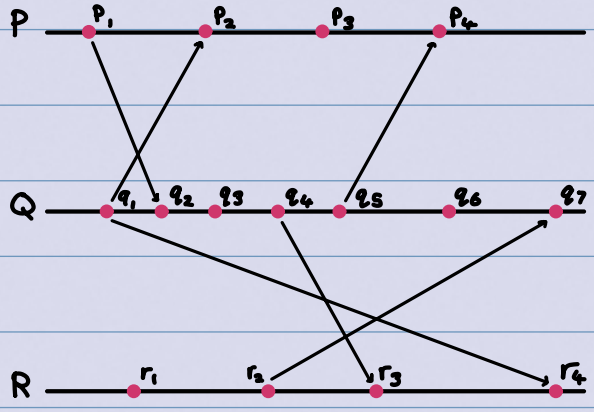
Still, any of the following may happen:



Events in a Distributed System

Given a distributed system of **processes** P, Q, R, ..., each process P consists of **events** P_1, P_2, ...

Example:



- event p_i in process P happened before event p_{i+1}
- if p_i is an event that sends a message to Q, then there is some event q_j in Q that receives this message, and p_i happened before q_j.

If an event p happened before an event q, then $p \to q$.

Observe:

- $\to$ is partial (neither $p \to q$ or $q \to p$ may hold)
- $\to$ is irreflexive ($p \to p$ never holds)
- $\to$ is transitive ($p \to q \wedge q \to r \Rightarrow p \to r$).
- $\to$ is asymmetric (if $p \to q$ then $\neg(q \to p)$).

$\rightsquigarrow$ the $\to$ is a strict partial order.

Also, let $p \not\to q$ abbreviate $\neg(p \to q)$.

1.4 - Logical Clocks

Two distinct events are said to be concurrent if $p \not\to q$ and $q \not\to p$.

↳ in the previous example, $p_1 \to r_4$ so they're not concurrent, but p_3 and q_3 are concurrent since $p_3 \not\to q_3$ and $q_3 \not\to p_3$.

Event Ordering: use a Logical Clock C, ie, C assigns a global timestamp $C(p)$ to each event p.

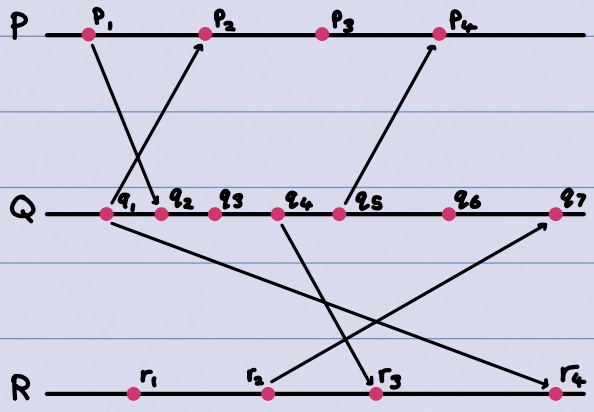
Function C satisfies the clock condition if, for any events p and q, $p \to q \Rightarrow C(p) < C(q)$.

For a distributed system, the clock condition holds if and only if:

1. p_i and p_j are events of P and $p_i \to p_j$ then $C(p_i) < C(p_j)$
2. p is the sending message by process P and q is the reciever of this message in process Q then $C(p) < C(q)$.

- a logical clock C with unique timestamps that satisfies the clock condition describes a total order $a < b$ (with $C(a) < C(b)$) that embeds the strict partial order $\to$
- the set defined by all those C is exactly the set of sequential executions possible to observe in the system.

Example: defining C satisfying the Clock Condition



One possible solution:

e	p_1	p_2	p_3	p_4
C(e)	1	4	7	12

e	q_1	q_2	q_3	q_4	q_5	q_6	q_7
C(e)	2	3	5	6	11	13	14

e	r_1	r_2	r_3	r_4
C(e)	8	9	10	15

1.5 - Cache Coherence

Idea: use happened-before diagrams to model more relaxed memory models

Given a path through each of the threads of a program:

- consider the actions of each thread as events of a process
- use more processes to model more memory

$\rightsquigarrow$ concisely represent some interleavings $\Rightarrow$ we establish models for memory coherence

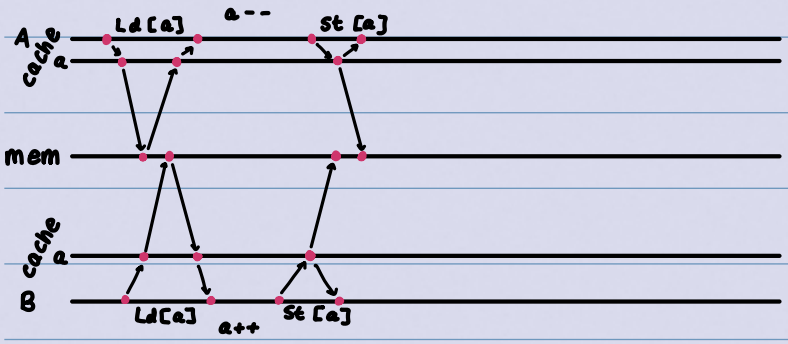
Weakening the Model: Introducing Caches

Observation: programs often access the same memory area repeatedly

Idea: processors use quickly accessible caches to avoid a costly round-trip to RAM for each memory access.

Model: each cache line as one process

Example: program that decrements a



issue here: process B has no effect here since the replicated a in the cache from process A overwrote process B's changes!

Cache Coherence:

1. Memory operations in program order ($\leq$) are embedded into the memory order ($\sqsubseteq$).

$$Op_i[a] \leq Op_i[a]' \Rightarrow Op_i[a] \sqsubseteq Op_i[a]'$$

2. For each address a, memory order $\sqsubseteq$ is a total order on all accesses to a
3. A load's value is determined by the latest store with respect to memory order: $val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\sqsubseteq}(\{St_k[a] \mid St_k[a] \sqsubseteq Ld_i[a]\})$

With:

- $Op_i[a]$: any memory access to address a by CPU i
- $Ld_i[a]$: a load from address a by CPU i
- $St_i[a]$: a store from address a by CPU i
- Program order $\leq$ being specified by the control flow of the program for each CPU

1.6 - Sequential Consistency

Messaging Protocol Example:

Thread A

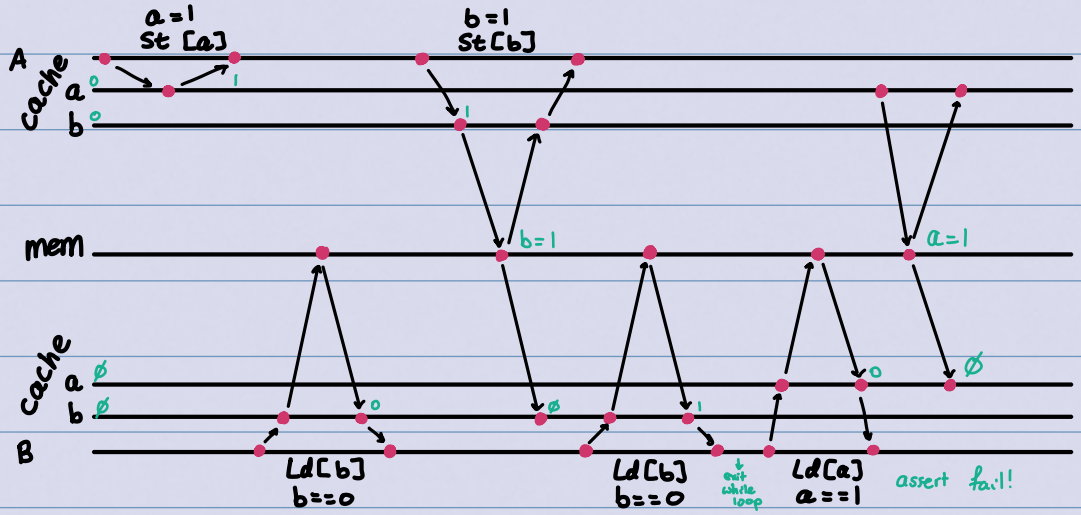
```
void foo(void) {
    a = 1;
    b = 1;
}
```

Thread B

```
void bar(void) {
    while (b != 1) {};
    assert a == 1;
}
```

- in all examples, the initial value of variables is assumed to be 0
- Suppose that a and b reside in different cache line
- assume that a cache line is larger than the variable itself

initially: each cache line one process, A caches a=0, b=0, and B's cache is empty



In this example, we can see a coherent cache but sequential inconsistency!

Sequential Consistency Conditions: the result of any execution is the same as if the memory operations

1. of all processors joined were executed in some sequential order
2. and the operations of each individual processor appear in this sequence in the order specified by its program

Idea for showing that a system is NOT sequentially consistent:

- pick a result obtained from a program run on a sequentially consistent system

- pick an execution ② and a total ordering of all operations ①
- add extra processes to model other system components
- show that a different total order $\sqsubseteq'$ that embeds $\leq$ exists, for which the result differs.

1.7 - Sequential Consistency (Formal Definition)

Sequential Consistency:

1. Memory operations in program order ($\leq$) are embedded into the total memory order ($\sqsubseteq$)

$$Op_i[a] \leq Op_i[b]' \Rightarrow Op_i[a] \sqsubseteq Op_i[b]'$$

2. A load's value is determined by the latest store wrt memory order

$$val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\sqsubseteq}(\{St_k[a] \mid St_k[a] \sqsubseteq Ld_i[a]\})$$

↳ this definition looks close to the definition of basic coherence! the difference is that the embedding part covers all memory locations instead of particular ones!

→ for sequential consistency, caches and memory need to collaborate in a Cache Coherence Protocol.

Benefits of Sequential Consistency

- concisely represent all interleavings that are due to variations in timing
- synchronisation using time is uncommon for software
- a good model for correct behaviors of concurrent programs

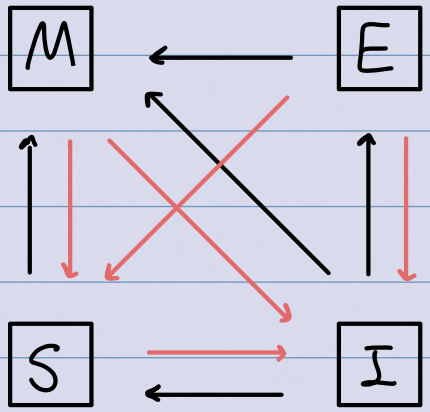
↳ suitable for concurrent processors that acquire exclusive access to memory

1.8 - Introduction to MESI Protocol

Problem: Keeping track of the latest copy of a value in distributed

local mirror images

Idea: attach a state machine to each cached line



Each cache line is in one of the states:

I: it's invalid and ready for reuse

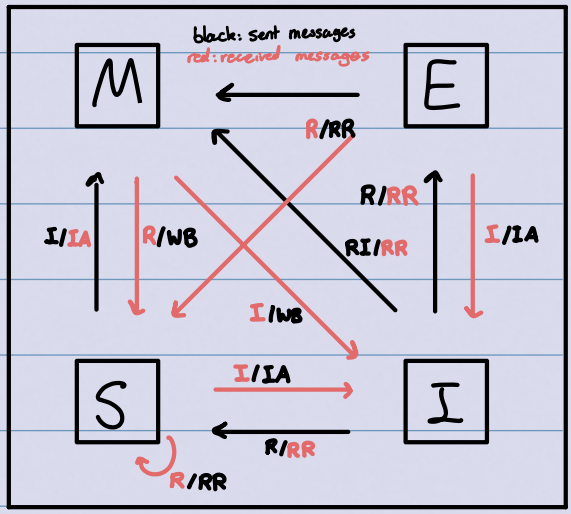
S: other caches have an identical copy; it's shared

E: the content is in no other cache; it's exclusive

M: the content is exclusive to this cache and has furthermore been modified

↳ the global state of cache lines is kept consistent by sending messages

Moving data between caches is coordinated by sending messages



R: Read → sent if CPU needs to load from an address

RR: Read Response → when in state E or S, reply to R, carries the data for the requested address

I: Invalidate → ask others to evict a cache line

IA: Invalidate Acknowledged → reply to I indicating that a cache line was evicted

RI: Read Invalidate → like Read + Invalidate (also called "read with intent to modify")

WB: Writeback → Special RR – when in state M, reply to R and as a side effect notifying main memory about modifications to the cache line

we mostly consider messages between processors. Upon read invalidate, a processor replies with RR / WB before the IA is sent.

1.9 – MESI Example

we'll write the content of a cache line as:

- M_x: modified, with value x
- E_x: Exclusive, with value x
- S_x: shared, with value x
- I: Invalid

Example:

Thread A

```
void foo(void) {
    a = 1;      // A.1
    b = 1;      // A.2
}
```

Thread B

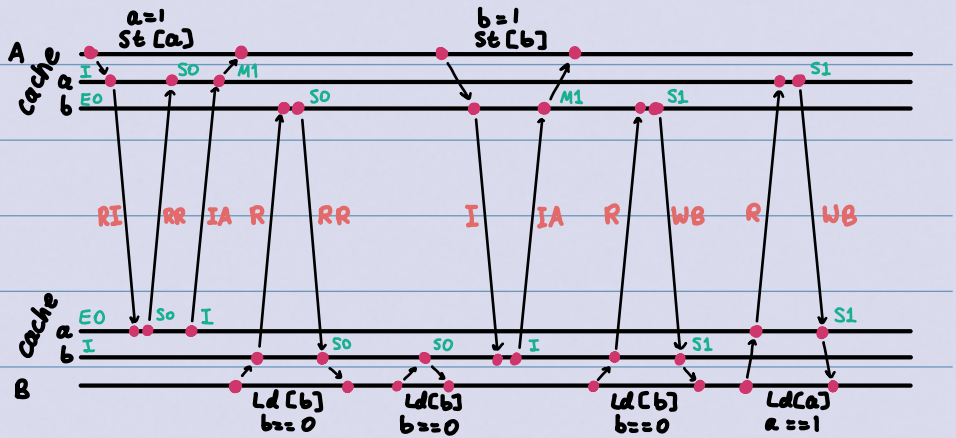
```
void bar(void) {
    while (b != 1) {};   // B.1
    assert a == 1;       // B.
}
```

Statement	CPU A a	CPU A b	CPU B a	CPU B b	RAM a	RAM b	message
A.1	I	I	I	I	0	0	
	I	I	I	I	0	0	↓ read invalidate of a from CPU A
	I	I	I	I	0	0	↓ invalidate ack. of a from CPU B
B.1	M1	I	I	I	0	0	↓ read response of a=0 from RAM
	M1	I	I	I	0	0	↓ read of b from CPU B
B.1	M1	I	I	E0	0	0	↓ read response with b=0 from RAM
A.2	M1	I	I	E0	0	0	
	M1	I	I	E0	0	0	↓ read invalidate of b from CPU A
	M1	S0	I	S0	0	0	↓ read response of b=0 from CPU B
B.1	M1	M1	I	I	0	0	↓ invalidate ack. of b from CPU B
	M1	M1	I	I	0	0	↓ read of b from CPU B
B.2	M1	S1	I	S1	0	1	↓ write back of b=1 from CPU A
	M1	S1	I	S1	0	1	↓ read of a from CPU B
	S1	S1	S1	S1	1	1	↓ write back of a=1 from CPU A

1.10 - MESI in the context of happened-before

Example:

- each cache line one process
- A caches b=0 as E
- B caches a=0 as E



Observations:

- each memory access must complete before executing next instruction ⇝ add edge
- second execution of test b==0 stays within cache ⇝ no traffic to mem / bus!!

Chapter 1 Summary: MESI Cache Coherence Protocol

Cache Coherence:

- A cache coherent system must appear to execute all threads' loads and stores to a **single memory location** in a total order that respects the program order of each thread
  ↳ memory order can still be partial!
- **MESI cache coherence protocol ensures cache coherence** for processors with caches

Sequential Consistency

- specifies that the system must appear to execute all threads' loads and stores to **all memory locations** in a total order that respects the program order of each thread
- a characterisation of well-behaved programs
- a model for differing speed of execution
- for fixed paths through the threads and a total order between accesses to the same variables: executions can be illustrated by a happened-before diagram with one process per cached variable
- **MESI cache coherence protocol ensures sequential consistency** for processors with simple caches.

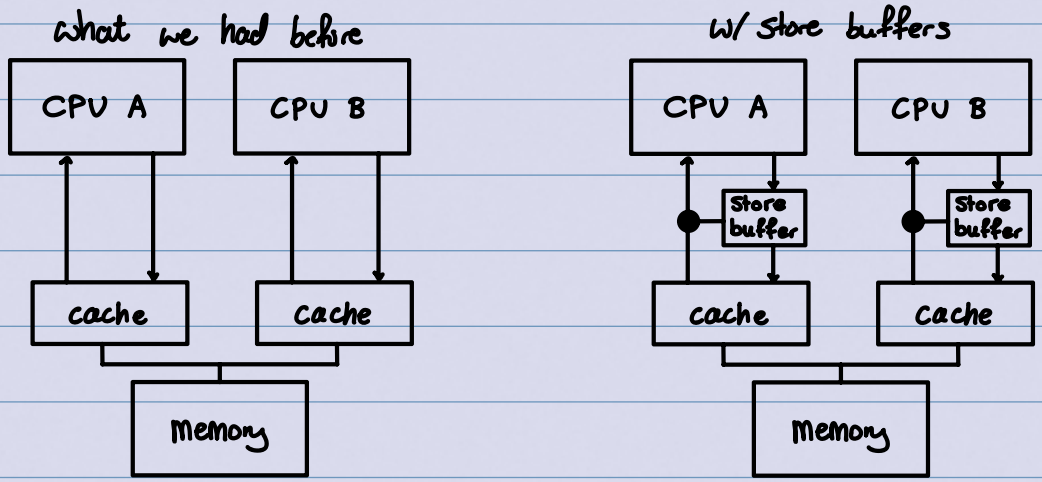
1.11 - Total Store Order

Out-of-Order Execution: **Stores always stall!** Since the CPU cannot

write until it owns the line, and getting ownership needs a round trip to main memory or the other caches.

↓

Store Buffers



- put each **store into a store buffer** and **continue execution**
- Store buffers apply stores in various orders (FIFO[TSO] / unordered[PSO] / etc).
- program order still needs to be observed locally!
  ↳ Store buffer snoops read channel and on matching address, **return the youngest value in the buffer.**

Definition: Total Store Order (TSO)

1. the **store order** with respect to memory ($\sqsubseteq$) is **total**

$$\forall_{a,b \in addr\ i,j \in CPU} (St_i[a] \sqsubseteq St_j[b]) \vee (St_j[b] \sqsubseteq St_i[a])$$

2. **stores in program order** ($\leq$) are **embedded into the memory order** ($\sqsubseteq$)

$$St_i[a] \leq St_j[b] \Rightarrow St_i[a] \sqsubseteq St_j[b]$$

3. **loads preceding an other operation** (with respect to program order $\leq$) are **embedded into the memory order** ($\sqsubseteq$)

$$Ld_i[a] \leq Op_i[b] \Rightarrow Ld_i[a] \sqsubseteq Op_i[b]$$

4. a load's value is determined by the latest store as observed by the local CPU

$$val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\sqsubseteq}(\{St_k[a] \mid St_k[a] \sqsubseteq Ld_i[a]\} \cup \{St_i[a] \mid St_i[a] \leq Ld_i[a]\}))$$

Particularly, one ordering property from SC is not guaranteed:

$$St_i[a] \leq Ld_i[b] \not\Rightarrow St_i[a] \sqsubseteq Ld_i[b]$$

↳ local stores may be observed earlier by local loads then from somewhere else!

Example: happened-before model for TSO

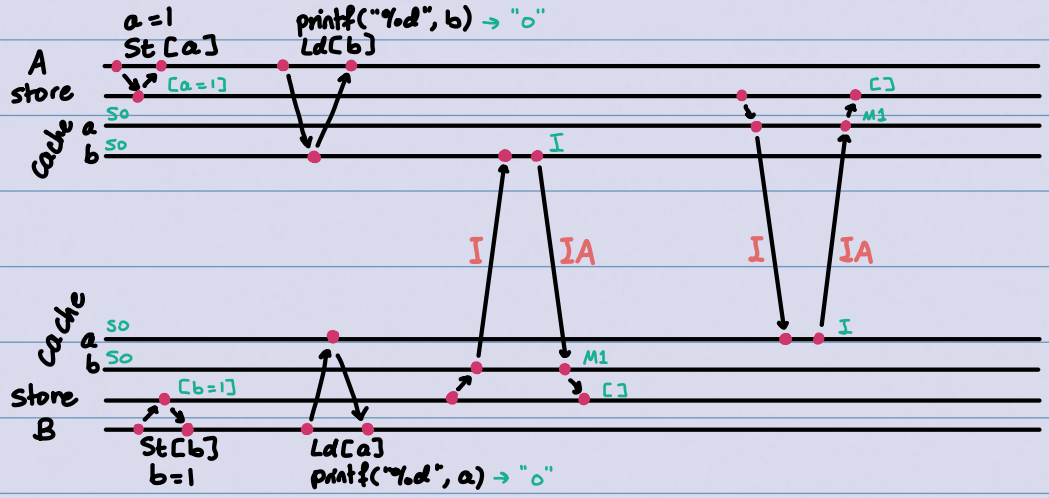
Thread A

```
a = 1;
printf("%d", b);
```

Thread B

```
b = 1;
printf("%d", );
```

assume cache A contains: a: S0, b: S0, and cache B contains: a: S0, b: S0



note: modern x86 CPUs provide the mfence operation which orders all memory operations as: $Op_i \leq mfence() \leq Op_i' \Rightarrow Op_i \sqsubseteq Op_i'$.

1.12 - Partial Store Order

Definition: Partial Store Order (PSO)

1. the store order with respect to memory ($\sqsubseteq$) is total

$$\forall_{a,b \in addr,\ i,j \in CPU} (St_i[a] \sqsubseteq St_j[b]) \vee (St_j[b] \sqsubseteq St_i[a])$$

2. fenced stores in program order ($\leq$) are embedded into the memory order ($\sqsubseteq$)

$$St_i[a] \leq sfence() \leq St_i[b] \Rightarrow St_i[a] \sqsubseteq St_i[b]$$

3. stores to the same address in program order ($\leq$) are embedded into the memory order ($\sqsubseteq$)

$$St_i[a] \leq St_i[a]' \Rightarrow St_i[a] \sqsubseteq St_i[a]'$$

4. loads preceding an other operation (with respect to program order $\leq$) are embedded into the memory order ($\sqsubseteq$)

$$Ld_i[a] \leq Op_i[b] \Rightarrow Ld_i[a] \sqsubseteq Op_i[b]$$

5. a load's value is determined by the latest store as observed by the local CPU

$$val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\sqsubseteq}(\{St_k[a] \mid St_k[a] \sqsubseteq Ld_i[a]\} \cup \{St_i[a] \mid St_i[a] \leq Ld_i[a]\}))$$

→ now also stores are not guaranteed to be in order anymore! :

$$St_i[a] \leq St_i[b] \not\Rightarrow St_i[a] \sqsubseteq St_i[b]$$

→ note: TSO is FIFO to leave store buffer, PSO is unordered. That's the only difference! ü

Example: happened-before model for PSO

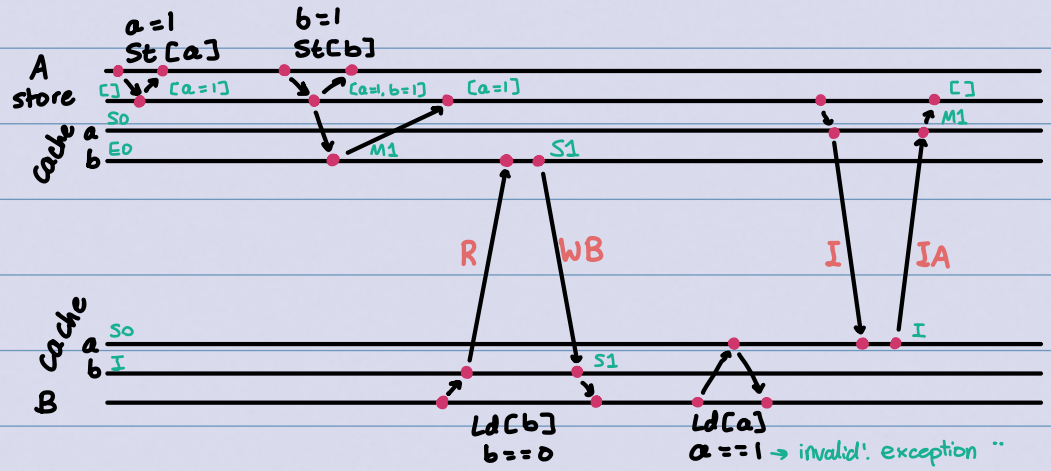
Thread A

```
a = 1;
b = 1;
```

Thread B

```
while (b==0) {}
assert (a==1);
```

assume cache A contains: a: S0, b: E0, and cache B contains: a: S0, b: I



↳ So, whenever a store in front of another operation in one CPU must be observable in this order by a different CPU, we must insert an explicit write barrier!

- a store barrier marks all current store operations in the store buffer
- the next store operation is only executed when all marked stores in the buffer have completed.

Example: Happened-Before Model for Store Barriers ⑤

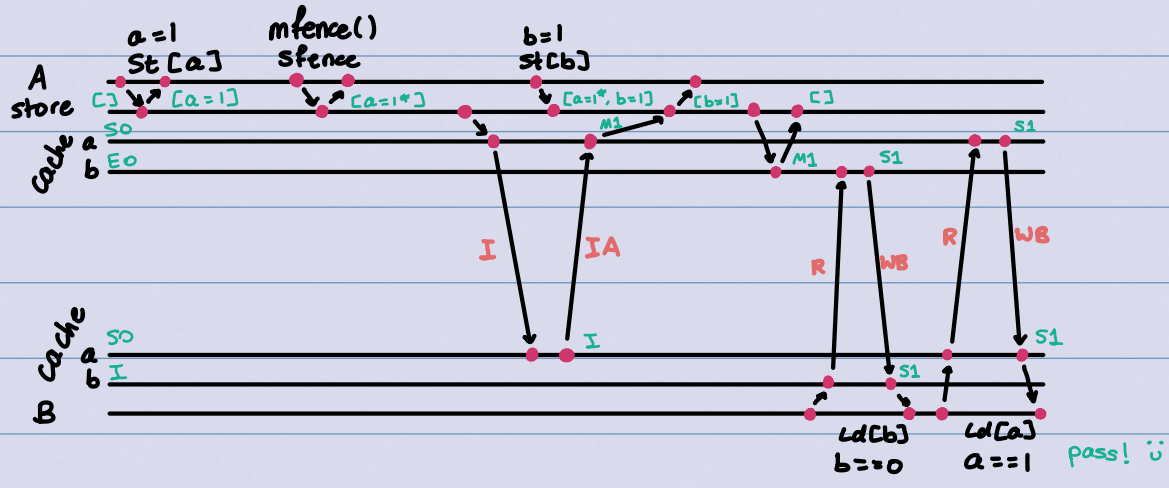
Thread A

```
a = 1;
sfence(); //!
b = 1;
```

Thread B

```
while (b==0) {}
assert (a==1);
```

assume cache A contains: a: S0, b: E0, and cache B contains: a: S0, b: I

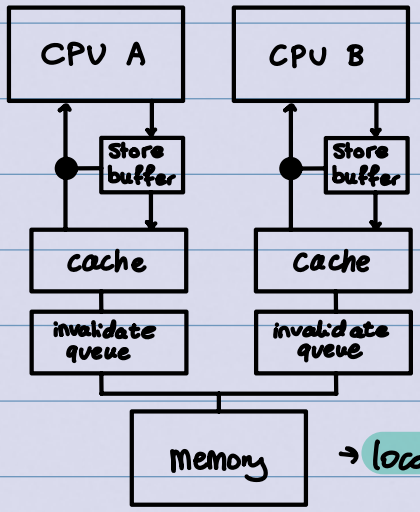


1.13 - Invalidate Queues

Communication of cache updates is still costly:

- a cache-intense computation can still fill up store buffers in CPUs
- invalidation acknowledgements are delayed on busy caches

Relaxed Memory Order (1/2): Invalidate Queues



→ immediately acknowledge an invalidation & apply it later

- put each invalidate message into an invalidate queue
- if a MESI message needs to be sent regarding a cache line in the invalidate queue then wait until the line is invalidated

→ local loads and stores do NOT consult the invalidate queue!

Example: Happened-Before Model for Invalidate Queues ⑥

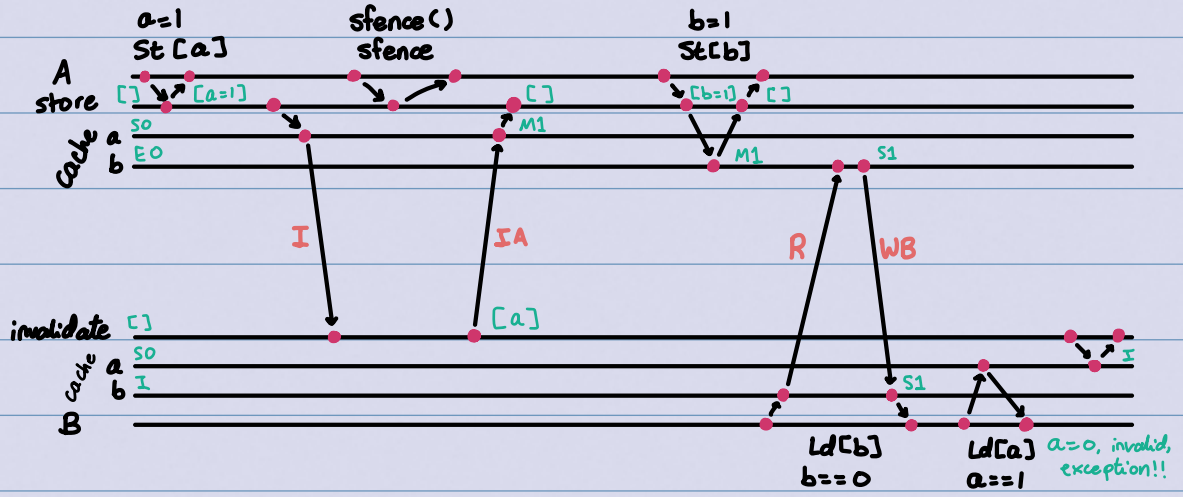
Thread A

```
a = 1;
sfence(); // !
b = 1;
```

Thread B

```
while (b==0) {}
assert (a==1);
```

assume cache A contains: a: S0, b: E0, and cache B contains: a: S0, b: I



Synchronisation with Invalidate Queues: Load Barriers

Load accesses do not consult the invalidate queue (being on the other side of the cache)

- Might load an out-of-date value
- need a way to establish sequential consistency between writes of other processors and local reads
- insert an explicit load barrier before a load access
  - ↳ a load barrier marks all entries in the invalidate queue
  - ↳ the next load operation is only executed once all marked invalidations have completed
- a load barrier before each load gives sequentially consistent load behavior (and is as slow as a system without invalidate queues)

~ match each store barrier in one process with a load barrier in another process

Example: Invalidate Queues with Load Barriers ⑦

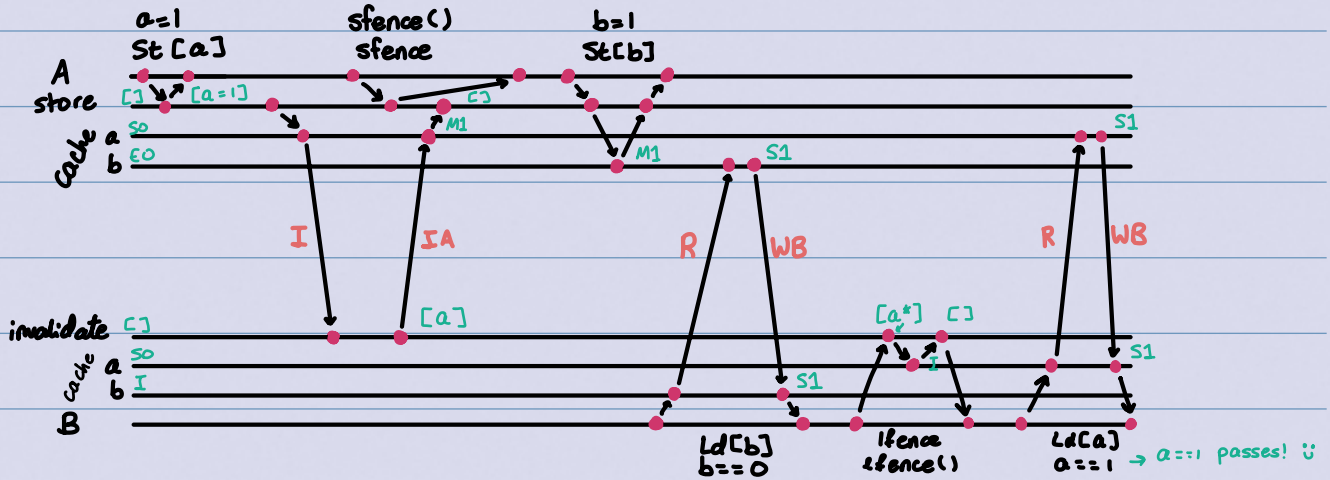
Thread A

```
a = 1;
sfence();
b = 1;
```

Thread B

```
while (b==0) {}
lfence(); //!
assert (a==1);
```

assume cache A contains: a: S0, b: E0, and cache B contains: a: S0, b: I

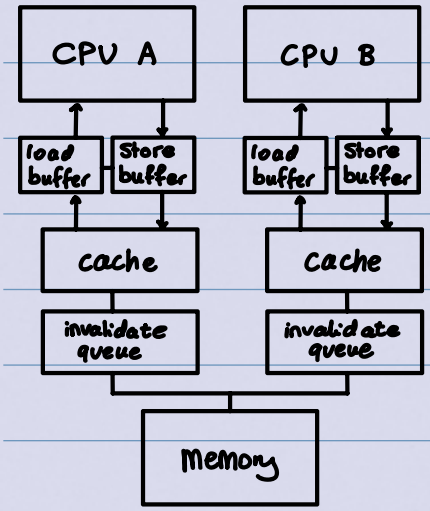


1.14 - Load Buffers

Communication of cache updates is still costly:

- performing stores is now deferred ✓
- but, cache missing loads still stall computation while going to/from memory

Relaxed Memory Order (2/2): Load Buffers



→ immediately return from loads!

- put each load instruction into a load buffer
- if the load's target register is used in a later instruction then wait until the load is performed
- loads also snoop the local store buffers!

Example: Happened-Before for Load Buffers ⑧

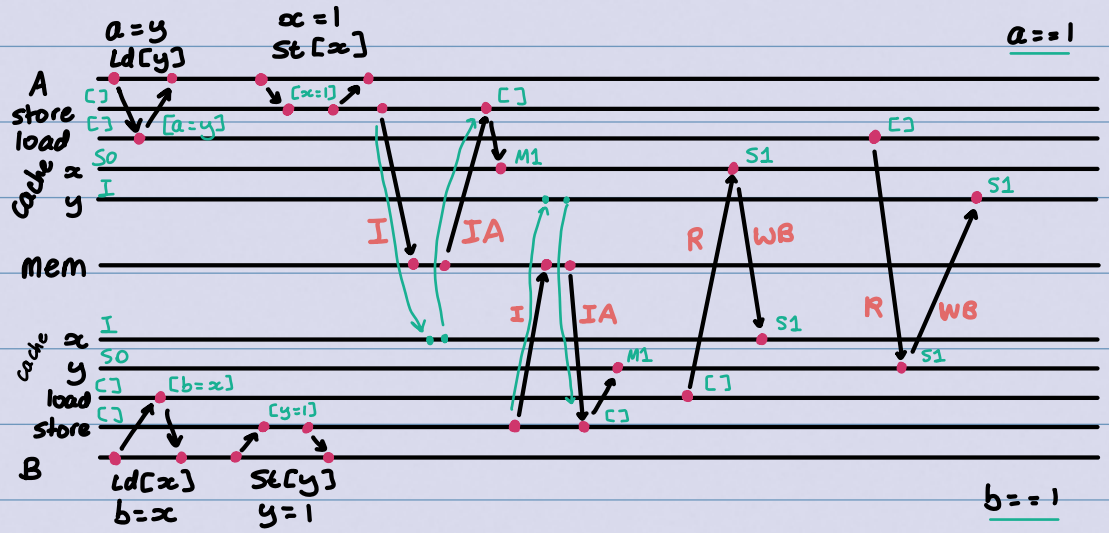
Thread A

```
a = y;
x = 1;
assert(a == 1);
```

Thread B

```
b = x;
y = 1;
assert(b == 1);
```

assume cache A contains: x: S0, y: I, and cache B contains: x: I, y: S0



Synchronisation with Load Buffers: Load Barriers

Load Accesses do not necessarily need to complete before the program continues!

- might lead to surprising loads "from the future"
- need a way to reliably synchronise program order with memory order
- insert an explicit load barrier after the load operation
  - ↳ a load barrier marks all entries in the load buffer
  - ↳ the next operation is only executed after all marked loads have completed
- a load barrier after each load gives sequentially consistent load behavior (and is as slow as a system without load buffers)

↪ match each store barrier in one process with a load barrier in another process

1.15 - Relaxed Memory Order

Definition: Relaxed Memory Order

1. Fenced memory access in program order ($\leq$) are embedded into the memory order ($\sqsubseteq$)

$$Op_i[a] \leq mfence() \leq Op_i[b] \Rightarrow Op_i[a] \sqsubseteq Op_i[b]$$

2. Stores to the same address in program order ($\leq$) are embedded into the memory order ($\sqsubseteq$)

$$Op_i[a] \leq St_i[a]' \Rightarrow Op_i[a] \sqsubseteq St_i[a]'$$

3. Operations dependant on a load (with respect to dependence $\rightarrow$) are embedded into the memory order ($\sqsubseteq$)

$$Ld_i[a] \rightarrow Op_i[b] \Rightarrow Ld_i[a] \sqsubseteq Op_i[b]$$

4. a load's value is determined by the latest store as observed by the local CPU

$$val(Ld_i[a]) = val(St_j[a] \mid St_j[a] = \max_{\subseteq}(\{St_k[a] \mid St_k[a] \subseteq Ld_i[a]\} \cup \{St_i[a] \mid St_i[a] \leq Ld_i[a]\}))$$

But, now we need the notion of dependence ($\rightarrow$):

- register loads are dependant on latest register stores:

$$Ld_i[a] = \max_{\leq}(Ld_i[a]' \mid targreg(Ld_i[a]') = srcreg(St_i[b] \wedge Ld_i[a]' \leq St_i[b]) \Rightarrow Ld_i[a] \rightarrow St_i[b]$$

- Stores within branched blocks are dependant on branch conditionals:

$$(Ld_i[a] \leq St_i[b]) \wedge Ld_i[a] \rightarrow condbranch \leq St_i[b] \Rightarrow Ld_i[a] \rightarrow St_i[b]$$

- Memory accesses to the same address:

$$St_i[a] \leq Ld_i[a] \Rightarrow St_i[a] \rightarrow Ld_i[a]$$

1.16 - Dekker's Algorithm and Summary

Mutual Exclusion of two processes with busy waiting:

```
flag[0] = false;   } flag[i] being true means process i wants
flag[1] = false;   } to enter the critical section.
turn = 0;  // process 0's turn rn
```

Process 0:

```
flag[0] = true;
while (flag[1] == true) {
    if (turn != 0) {
        flag[0] = false;      // busy wait
        while (turn != 0) { }
        flag[0] = true;
```

Process 1:

```
flag[1] = true;
while (flag[0] == true) {
    if (turn != 1) {
        flag[1] = false;      // busy wait
        while (turn != 1) { }
        flag[1] = true;
```

```
    }
}
// critical section!
turn = 1;
flag[0] = false;
```

```
    }
}
// critical section!
turn = 0;
flag[1] = false;
```

The Idea Behind Dekker

Communication via 3 variables:

- `flag[i] == true` : process P_i wants to enter it's critical section
- `turn == i` : process P_i has priority when both want to enter

in process P_i:

- if P_{1-i} doesn't want to enter, can proceed directly into the critical section

↝ flag[i] is a lock and may be implemented as such

- if P_{1-i} also wants to enter, wait for turn to be set to i
- while waiting for turn, reset flag[i] to enable P_{1-i} to progress

Dekker's Algorithm and RMO:

Problem: Dekker's algorithm requires sequential consistency

Idea: insert memory barriers between all variables common to both threads.

Process 0

```
flag[0] = true;
sfence();
while (lfence() && flag[1] == true) {
    if (lfence() && turn != 0) {
        flag[0] = false;
```

- insert a load memory barrier lfence() in front of every read of common variables

```
        sfence();
        while (lfence() && turn != 0) { }
        flag[0] = true;
        sfence();
    }
}
// critical section!
turn = 1;
sfence();
flag[0] = false;
sfence();
```

busy wait

- insert a store memory barrier sfence() after writing a variable that's read in the other thread
- the lfence() of the first iteration of each loop may be combined with the preceding sfence() to an mfence()

Summary: Relaxed Memory Models

Highly optimised CPUs may use a relaxed memory model:

- loads and stores are not synchronised unless requested by the user
- many kinds of memory barriers exist with subtle differences

↝ memory barriers are the "lowest-level" of synchronisation.

1.17 - Cache Consistency Summary

Memory barriers reside at the lowest level of synchronisation primitives.

Where are they useful?

- when blocking should not de-schedule threads
- when several processes implement automata and coordinate their transitions via common synchronised variables

When might they not be appropriate?

- difficult to get right, best suited for specific, well-understood algorithms

- often synchronisation with locks is as fast and easier
- too many fences are costly if store/invalidate buffers are the bottleneck

Standard Program optimisations include loop-invariant code motion and dead store elimination.

↳ Example:

before optimisation

```
int x = 0; // dead store
for (int i=0; i<100; i++) {
    x = 1; // loop-invariant
    printf("%d", x);
}
```

after optimisation

```
int x = 1;
for (int i=0; i<100; i++) {
    printf("%d", x);
}
```

Compilers may also reorder store instructions!

- store barriers keep the compiler from reordering across
- the specification of volatile keeps the C compiler from reordering memory accesses to this address

↳ Example:

before optimisation

```
int x = 0;
for (int i=0; i<100; i++) {
    sfence(); //!
    x = 1;
    printf("%d", x);
}
```

after optimisation

```
volatile int x = 0;
for (int i=0; i<100; i++) {
    x = 1;
    printf("%d", x);
}
```